# Bot Games

Exact creation fields, REST endpoints, socket events, and client flow.

## Overview

A bot game is created through an authenticated REST request. The returned bot-game ID is then used to open an authenticated WebSocket. Human moves and resignations are sent through the socket; bot state and thinking events are received from it.

## Create bot request

Source: data/remote/dto/CreateBotGameRequest.kt

| Field | Type | JSON name | Meaning |
|---|---|---|---|
| level | String | level | Bot difficulty, for example easy or custom. |
| rating | Int? | rating | Required for custom-level rating; otherwise null. |
| color | String | color | Human player color preference. |

```
POST /api/bot-games Body: { level, rating, color }
```

## Bot game response

Source: data/remote/dto/BotGameDto.kt

| Field | Type | JSON name | Meaning |
|---|---|---|---|
| id | String | id | Bot game ID. |
| playerId | Long | player_id | Human player ID. |
| playerColor | String | player_color | Human color assigned by server. |
| botLevel | String | bot_level | Selected bot difficulty. |
| botRating | Int | bot_rating | Bot rating. |
| position | String | position | Current board position/FEN-like value. |
| moves | List<String> | moves | Moves already played. |
| status | String | status | Game status. |
| result | String? | result | Game result when finished. |
| endReason | String? | end_reason | Why game ended. |
| createdAt | String | created_at | Creation timestamp. |
| endedAt | String? | ended_at | Finish timestamp. |

## Creation flow

```
BotSetupScreen -> BotSetupViewModel.startGame() -> BotGameRepository.createGame(level, rating, color) -> BotGameService.createBotGame() -> BotGameDto.id -> navigate bot_game/{gameId} -> BotGameViewModel opens BotSocketClient
```

## Bot REST endpoints

| HTTP | Path | Method | Use |
|---|---|---|---|
| POST | api/bot-games | createBotGame | Create a bot game. |
| GET | api/bot-games/{gameID} | getBotGame | Load bot-game details. |

## Bot WebSocket outgoing events

Source: data/remote/dto/BotSocketOutgoing.kt

| Event | Fields | Meaning |
|---|---|---|
| move | move: String | Send a human move in the server move format. |
| resign | none | Resign the bot game. |

## Bot WebSocket incoming events

Source: data/remote/dto/BotSocketIncoming.kt

| Event | Fields | Meaning |
|---|---|---|
| game_state | game: BotGameDto | Full bot game state. |
| bot_thinking | none | Bot is calculating. |
| error | message: String | Server/socket error. |

## Authentication

BotGameRepository creates BotGameService through RetrofitInstance.createBotGameService(TokenManager(context)), so REST creation and loading use the shared interceptor and authenticator. BotSocketClient calls AuthRepository.isLoggedIn() before reading the current access token and opening the socket at /ws/bot-games/{gameId}?token={accessToken}.

## Files

| File | Responsibility |
|---|---|
| presentation/game/viewmodel/BotSetupViewModel.kt | Stores level, custom rating, color, loading, error, and created ID. |
| presentation/game/screens/BotSetupScreen.kt | Bot setup UI. |
| presentation/game/BotGameViewModel.kt | Bot game state, moves, resign, socket event handling. |

| File | Responsibility |
| --- | --- |
| presentation/game/screens/BotGameScreen.kt | Bot game UI. |
| data/repository/BotGameRepository.kt | REST repository wrapper. |
| data/remote/BotGameService.kt | REST API contract. |
| data/remote/BotSocketClient.kt | WebSocket transport. |